ShopSmart

Quality and Deployment Guided Challenge — Final Submission

Project Information

Student	John Kerby Sambile
Project Name	ShopSmart
Repository	https://github.com/teptepphhh/COGNIX--ShopSmart-Challenge.git
Branch	ASC_AI_ASSISTANT_SHOPSMART_SAMBILE
Student ID	23-1708-191

1. Overview

ShopSmart is a classroom e-commerce starter application built with React, Express, Prisma, and PostgreSQL. It includes a product catalog, a shopping cart, a simulated checkout, and a grounded product assistant that recommends current in-stock items using catalog-based validation.

2. Quality Check Summary

The project was validated using the repository check command:

```
npm run check
```

Check	Result
Frontend lint	✔️ Passed
Backend lint	✔️ Passed
Frontend tests	✔️ Passed
Backend tests	✔️ Passed
Production build	✔️ Passed

All checks passed, confirming the system is stable and release-ready for the tested flows.

3. Functional Validation Results

Flow	Validated Behavior	Status
Home page	Landing page loads; product catalog is visible; search and category controls display as expected.	✔️ Pass
Cart	Add to cart works; cart count and total update; remove item works.	✔️ Pass
Checkout	Valid simulated checkout succeeds; order confirmation and correct total appear; cart clears after the order is placed.	✔️ Pass
AI Assistant	Valid questions return relevant recommendations; no-match requests return an honest no-result message; short/invalid questions return HTTP 422; a fallback response appears when provider output is unavailable or unverified.	✔️ Pass

4. Screenshot Evidence

Screenshot A — Catalog Match

Question asked: *"Which keyboard is below ₱2,000?"*

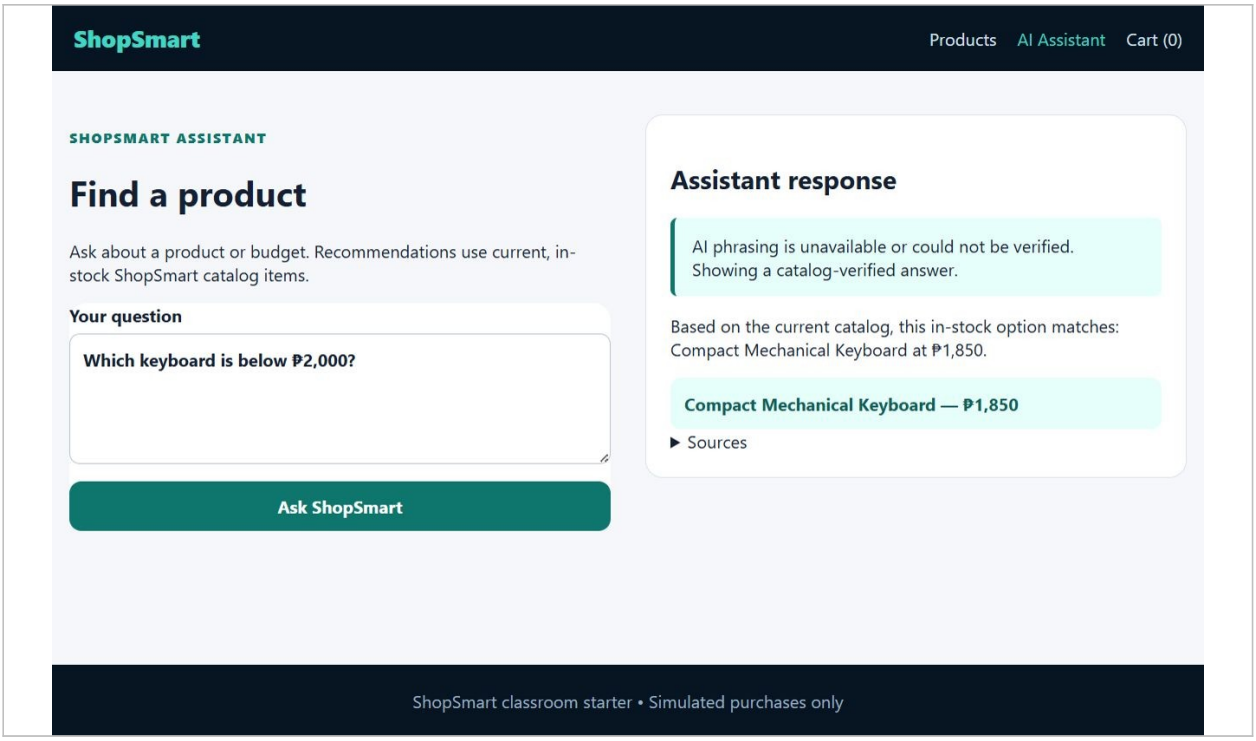


Figure: Screenshot A — Catalog Match

Observed result

- Compact Mechanical Keyboard was recommended.
- Price shown: ₱1,850.
- Product link is valid.
- Source is traceable to the selected product.

Screenshot B — No Match

Question asked: *"Do you sell refrigerators?"*

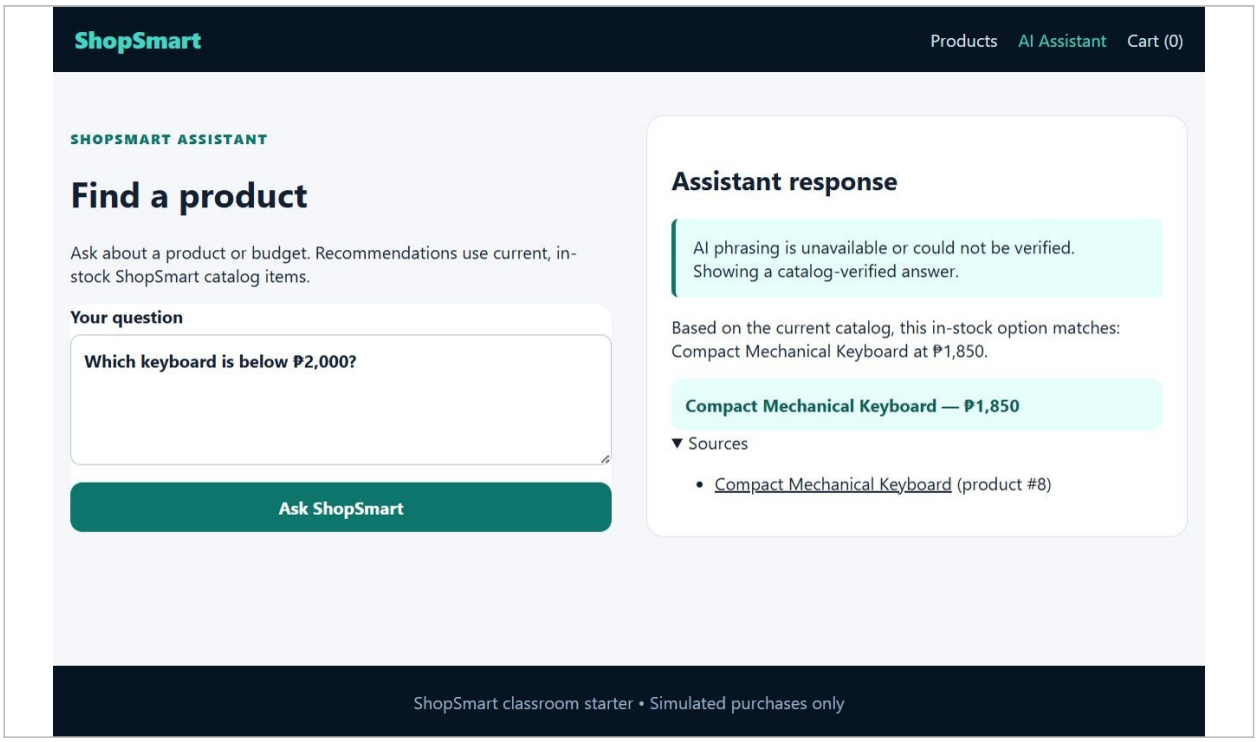


Figure: Screenshot B — No Match

Observed result

- An honest no-match response was returned.
- No invented products or sources were shown.

Screenshot C — Fallback / Provider Failure

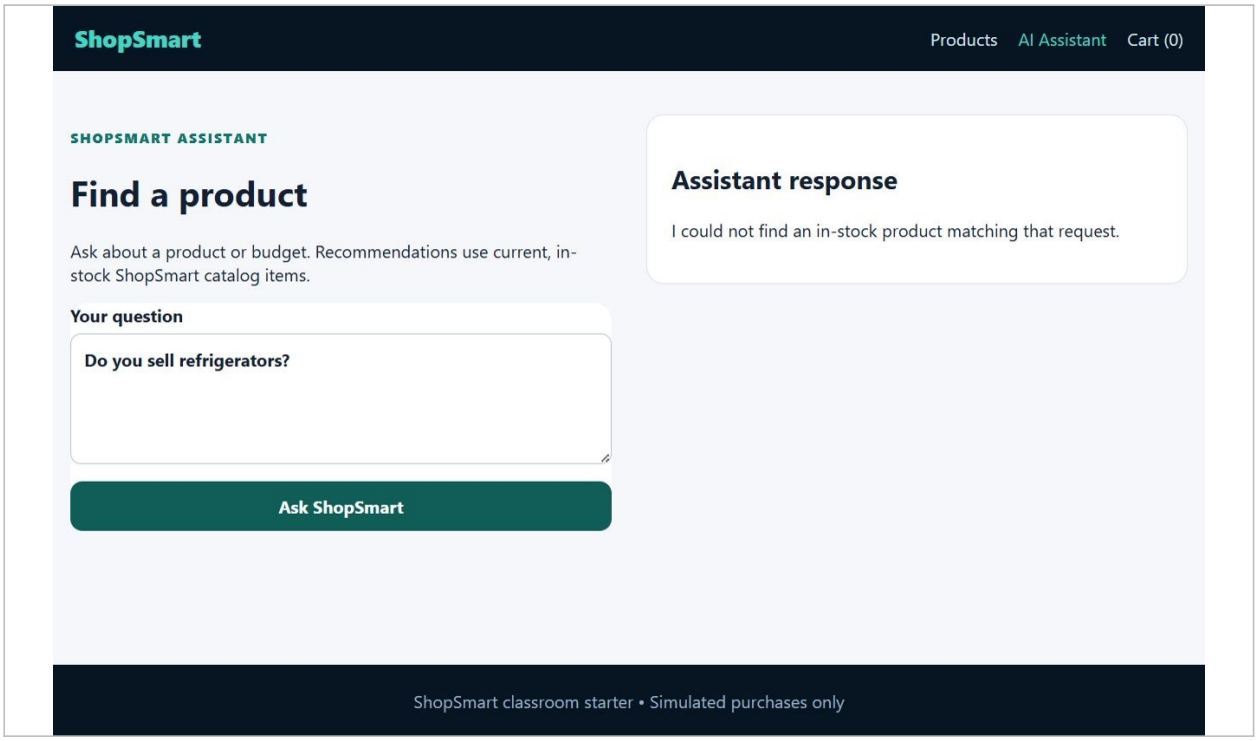


Figure: Screenshot C — Fallback / Provider Failure

Observed result

- A fallback notice appears when provider output is unavailable or cannot be verified.
- The catalog-backed answer remains valid and safe.

5. Deployment Notes

The application runs in a local development environment:

Component	Location
Frontend	http://localhost:5173
Backend	http://localhost:3000
Database	PostgreSQL via Prisma

Required configuration

Setting	Purpose
`DATABASE_URL`	PostgreSQL connection string (required)
`PORT, FRONTEND_URL, NODE_ENV`	Backend .env values (required)
`GEMINI_API_KEY, GEMINI_MODEL`	Provider-assisted AI responses (optional)
`Frontend API URL`	Must point to the deployed backend
`CORS settings`	Must allow production origins

6. Security and Data Handling

- No API keys or secrets were committed.
- No real customer information was used.
- No payment processing was performed.
- Only catalog data was used for AI recommendation logic.

7. Final Assessment

The project passed all required quality checks and validated the main user journeys across both the storefront and the AI assistant. Based on the evidence captured, the application is ready for guided challenge submission.

8. Submission Checklist

- ☑️ GitHub repository included
- ☑️ Branch included
- ☑️ Quality checks recorded
- ☑️ Screenshots included
- ☑️ Deployment notes included
- ☑️ No secrets or sensitive data included